

DBGS - PSIS

1.0

Generado por Doxygen 1.13.2

1 Índice de clases	1
1.1 Lista de clases	1
2 Índice de archivos	3
2.1 Lista de archivos	3
3 Documentación de clases	5
3.1 Referencia de la plantilla de la clase BPlusTree< T >	5
3.1.1 Descripción detallada	6
3.1.2 Documentación de constructores y destructores	6
3.1.2.1 BPlusTree()	6
3.1.2.2 ~BPlusTree()	7
3.1.3 Documentación de funciones miembro	7
3.1.3.1 _findIndex()	7
3.1.3.2 _findKey()	7
3.1.3.3 _findRange()	7
3.1.3.4 _innerInsert()	8
3.1.3.5 _insertInChild()	8
3.1.3.6 _insertInParent()	9
3.1.3.7 _insertItem()	9
3.1.3.8 _print()	9
3.1.3.9 _removeFromParent()	10
3.1.3.10 clear()	11
3.1.3.11 getRoot()	11
3.1.3.12 insert()	11
3.1.3.13 remove()	11
3.1.3.14 search()	12
3.1.3.15 searchRange()	12
3.2 Referencia de la estructura Column	13
3.2.1 Descripción detallada	13
3.3 Referencia de la plantilla de la clase HashMap< K, V >	13
3.3.1 Descripción detallada	14
3.3.2 Documentación de constructores y destructores	14
3.3.2.1 HashMap()	14
3.3.2.2 ~HashMap()	15
3.3.3 Documentación de funciones miembro	15
3.3.3.1 _getLoadFactor()	15
3.3.3.2 _getPows()	15
3.3.3.3 _hashCode()	15
3.3.3.4 _insert()	16
3.3.3.5 _toStringGeneric()	16
3.3.3.6 _wrap()	16
3.3.3.7 containsKey()	17

3.3.3.8 get()	17
3.3.3.9 getSize()	17
3.3.3.10 insert()	18
3.3.3.11 isEmpty()	18
3.3.3.12 remove()	18
3.3.3.13 toList()	18
3.4 Referencia de la plantilla de la clase HashNode< K, V >	19
3.4.1 Descripción detallada	19
3.4.2 Documentación de constructores y destructores	19
3.4.2.1 HashNode()	19
3.5 Referencia de la estructura IndexEntry	20
3.5.1 Descripción detallada	20
3.5.2 Documentación de funciones miembro	20
3.5.2.1 operator<()	20
3.5.2.2 operator<=()	20
3.5.2.3 operator==(())	21
3.5.2.4 operator>()	21
3.5.2.5 operator>=()	21
3.6 Referencia de la clase InvertedIndex	21
3.6.1 Descripción detallada	22
3.6.2 Documentación de constructores y destructores	22
3.6.2.1 InvertedIndex()	22
3.6.2.2 ~InvertedIndex()	22
3.6.3 Documentación de funciones miembro	22
3.6.3.1 add()	22
3.6.3.2 get()	22
3.6.3.3 remove()	23
3.7 Referencia de la plantilla de la estructura Node< T >	23
3.7.1 Descripción detallada	24
3.7.2 Documentación de constructores y destructores	24
3.7.2.1 Node()	24
3.8 Referencia de la clase Table	24
3.8.1 Descripción detallada	26
3.8.2 Documentación de constructores y destructores	26
3.8.2.1 Table()	26
3.8.2.2 ~Table()	26
3.8.3 Documentación de funciones miembro	26
3.8.3.1 cellMatchesType()	26
3.8.3.2 columnIndex()	26
3.8.3.3 createBPlusTreeIndex()	27
3.8.3.4 createInvertedIndex()	27
3.8.3.5 dataTypeToString()	27

3.8.3.6 <code>getCell()</code>	27
3.8.3.7 <code>insertRow()</code>	28
3.8.3.8 <code>searchByIndex()</code>	28
4 Documentación de archivos	29
4.1 Referencia del archivo <code>src/structures/BPlusTree.hpp</code>	29
4.1.1 Descripción detallada	29
4.2 <code>BPlusTree.hpp</code>	29
4.3 Referencia del archivo <code>src/structures/HashMap.hpp</code>	38
4.3.1 Descripción detallada	38
4.4 <code>HashMap.hpp</code>	39
4.5 <code>InvertedIndex.cpp</code>	41
4.6 Referencia del archivo <code>src/structures/InvertedIndex.hpp</code>	42
4.6.1 Descripción detallada	42
4.7 <code>InvertedIndex.hpp</code>	43
4.8 Referencia del archivo <code>src/system/IndexEntry.hpp</code>	43
4.8.1 Descripción detallada	43
4.9 <code>IndexEntry.hpp</code>	44
4.10 Referencia del archivo <code>src/system/Table.cpp</code>	44
4.10.1 Descripción detallada	44
4.11 <code>Table.cpp</code>	44
4.12 Referencia del archivo <code>src/system/Table.hpp</code>	47
4.12.1 Descripción detallada	47
4.13 <code>Table.hpp</code>	48
Índice alfabético	49

Capítulo 1

Índice de clases

1.1. Lista de clases

Lista de clases, estructuras, uniones e interfaces con breves descripciones:

BPlusTree< T >	Implementación genérica de un árbol B+	5
Column	Representa una columna de la tabla, con nombre y tipo de dato	13
HashMap< K, V >	Implementación genérica de un mapa hash con direccionamiento abierto y redimensionamiento	13
HashNode< K, V >	Nodo que almacena un par clave-valor en la tabla hash	19
IndexEntry	Entrada de índice para estructuras de índice (B+ o invertido)	20
InvertedIndex	Índice invertido basado en HashMap para asociar claves con listas de identificadores	21
Node< T >	Representa un nodo en el árbol B+	23
Table	Representa una tabla de datos con soporte para índices y operaciones básicas	24

Capítulo 2

Índice de archivos

2.1. Lista de archivos

Lista de todos los archivos documentados y con breves descripciones:

src/structures/ BPlusTree.hpp	
Implementación de un árbol B+ genérico	29
src/structures/ HashMap.hpp	
Implementación de un mapa hash genérico con manejo de colisiones y redimensionamiento automático	38
src/structures/ InvertedIndex.cpp	41
src/structures/ InvertedIndex.hpp	
Implementación de un índice invertido utilizando un mapa hash	42
src/system/ IndexEntry.hpp	
Definición de la estructura IndexEntry para índices de tablas	43
src/system/ Table.cpp	
Implementación de la clase Table	44
src/system/ Table.hpp	
Definición de la clase Table y estructuras auxiliares para la gestión de tablas de datos	47

Capítulo 3

Documentación de clases

3.1. Referencia de la plantilla de la clase BPlusTree< T >

Implementación genérica de un árbol B+.

```
#include <BPlusTree.hpp>
```

Métodos públicos

- **BPlusTree** (int _degree)
Constructor del árbol B+.
- **~BPlusTree** ()
Destructor del árbol B+.
- **Node**< T > * **_findKey** (**Node**< T > *node, T key)
Busca el nodo hoja que contiene la clave especificada.
- **Node**< T > * **_findRange** (**Node**< T > *node, T key)
Busca el nodo hoja donde debería estar la clave.
- int **_findIndex** (T *arr, T data, int len)
Encuentra el índice donde insertar un dato en un arreglo ordenado.
- **Node**< T > ** **_innerInsert** (**Node**< T > **child_arr, **Node**< T > *child, int len, int index)
Inserta un hijo en el arreglo de hijos de un nodo interno.
- T * **_insertItem** (T *arr, T data, int len)
Inserta un dato en el arreglo de items de un nodo.
- **Node**< T > * **_insertInChild** (**Node**< T > *node, T data, **Node**< T > *child)
Inserta un dato y un hijo en un nodo interno.
- void **_insertInParent** (**Node**< T > *par, **Node**< T > *child, T data)
Inserta un nuevo hijo en el nodo padre después de una división.
- void **_removeFromParent** (**Node**< T > *node, int index, **Node**< T > *par)
Elimina un hijo del nodo padre.
- void **_print** (**Node**< T > *cursor)
Imprime recursivamente el árbol desde el nodo dado.
- **Node**< T > * **getRoot** ()
Obtiene la raíz del árbol.
- int **searchRange** (T start, T end, T *result_data, int arr_length)
Busca todos los elementos en el rango [start, end].

- bool **search** (T data)
Busca si un elemento existe en el árbol.
- void **insert** (T data)
Inserta un elemento en el árbol.
- void **remove** (T data)
Elimina un elemento del árbol.
- void **clear** (Node< T > *cursor)
Libera la memoria de todos los nodos del árbol.
- void **print** ()
Imprime el contenido del árbol.

Atributos privados

- Node< T > * **root**
Puntero a la raíz del árbol B+.
- int **degree**
Grado del árbol (máximo número de hijos por nodo).

3.1.1. Descripción detallada

```
template<typename T>
class BPlusTree< T >
```

Implementación genérica de un árbol B+.

Esta clase implementa un árbol B+ que permite almacenar, buscar, eliminar y recorrer elementos de tipo T. El árbol B+ es una estructura de datos balanceada utilizada comúnmente en sistemas de bases de datos y sistemas de archivos.

Parámetros de plantilla

<i>T</i>	Tipo de dato de los elementos almacenados en el árbol.
----------	--

3.1.2. Documentación de constructores y destructores

3.1.2.1. BPlusTree()

```
template<typename T>
BPlusTree< T >::BPlusTree (
    int _degree)
```

Constructor del árbol B+.

Parámetros

<i>_degree</i>	Grado del árbol (máximo número de hijos por nodo).
----------------	--

3.1.2.2. ~BPlusTree()

```
template<typename T>
BPlusTree< T >::~~BPlusTree ()
```

Destructor del árbol B+.

Libera toda la memoria utilizada.

3.1.3. Documentación de funciones miembro**3.1.3.1. _findIndex()**

```
template<typename T>
int BPlusTree< T >::_findIndex (
    T * arr,
    T data,
    int len)
```

Encuentra el índice donde insertar un dato en un arreglo ordenado.

Parámetros

<i>arr</i>	Arreglo de datos.
<i>data</i>	Dato a insertar.
<i>len</i>	Longitud del arreglo.

Devuelve

Índice de inserción.

3.1.3.2. _findKey()

```
template<typename T>
Node< T > * BPlusTree< T >::_findKey (
    Node< T > * node,
    T key)
```

Busca el nodo hoja que contiene la clave especificada.

Parámetros

<i>node</i>	Nodo desde el cual iniciar la búsqueda.
<i>key</i>	Clave a buscar.

Devuelve

Puntero al nodo que contiene la clave, o nullptr si no se encuentra.

3.1.3.3. _findRange()

```
template<typename T>
Node< T > * BPlusTree< T >::_findRange (
    Node< T > * node,
    T key)
```

Busca el nodo hoja donde debería estar la clave.

Parámetros

<i>node</i>	Nodo desde el cual iniciar la búsqueda.
<i>key</i>	Clave a buscar.

Devuelve

Puntero al nodo hoja correspondiente.

3.1.3.4. _innerInsert()

```
template<typename T>
Node< T > ** BPlusTree< T >::_innerInsert (
    Node< T > ** child_arr,
    Node< T > * child,
    int len,
    int index)
```

Inserta un hijo en el arreglo de hijos de un nodo interno.

Parámetros

<i>child_arr</i>	Arreglo de hijos.
<i>child</i>	Nuevo hijo a insertar.
<i>len</i>	Longitud actual del arreglo.
<i>index</i>	Índice donde insertar el nuevo hijo.

Devuelve

Arreglo de hijos actualizado.

3.1.3.5. _insertInChild()

```
template<typename T>
Node< T > * BPlusTree< T >::_insertInChild (
    Node< T > * node,
    T data,
    Node< T > * child)
```

Inserta un dato y un hijo en un nodo interno.

Parámetros

<i>node</i>	Nodo interno.
<i>data</i>	Dato a insertar.
<i>child</i>	Hijo a insertar.

Devuelve

Nodo actualizado.

3.1.3.6. _insertInParent()

```
template<typename T>
void BPlusTree< T >::_insertInParent (
    Node< T > * par,
    Node< T > * child,
    T data)
```

Inserta un nuevo hijo en el nodo padre después de una división.

Parámetros

<i>par</i>	Nodo padre.
<i>child</i>	Nuevo hijo.
<i>data</i>	Dato promovido.

3.1.3.7. _insertItem()

```
template<typename T>
T * BPlusTree< T >::_insertItem (
    T * arr,
    T data,
    int len)
```

Inserta un dato en el arreglo de items de un nodo.

Parámetros

<i>arr</i>	Arreglo de items.
<i>data</i>	Dato a insertar.
<i>len</i>	Longitud actual del arreglo.

Devuelve

Arreglo de items actualizado.

3.1.3.8. _print()

```
template<typename T>
void BPlusTree< T >::_print (
    Node< T > * cursor)
```

Imprime recursivamente el árbol desde el nodo dado.

Parámetros

<i>cursor</i>	Nodo desde el cual imprimir.
---------------	------------------------------

3.1.3.9. `_removeFromParent()`

```
template<typename T>
void BPlusTree< T >::_removeFromParent (
    Node< T > * node,
    int index,
    Node< T > * par)
```

Elimina un hijo del nodo padre.

Parámetros

<i>node</i>	Nodo a eliminar.
<i>index</i>	Índice del hijo a eliminar.
<i>par</i>	Nodo padre.

3.1.3.10. clear()

```
template<typename T>
void BPlusTree< T >::clear (
    Node< T > * cursor)
```

Libera la memoria de todos los nodos del árbol.

Parámetros

<i>cursor</i>	Nodo desde el cual iniciar la liberación.
---------------	---

3.1.3.11. getRoot()

```
template<typename T>
Node< T > * BPlusTree< T >::getRoot ()
```

Obtiene la raíz del árbol.

Devuelve

Puntero a la raíz.

3.1.3.12. insert()

```
template<typename T>
void BPlusTree< T >::insert (
    T data)
```

Inserta un elemento en el árbol.

Parámetros

<i>data</i>	Elemento a insertar.
-------------	----------------------

3.1.3.13. remove()

```
template<typename T>
void BPlusTree< T >::remove (
    T data)
```

Elimina un elemento del árbol.

Parámetros

<i>data</i>	Elemento a eliminar.
-------------	----------------------

3.1.3.14. search()

```
template<typename T>
bool BPlusTree< T >::search (
    T data)
```

Busca si un elemento existe en el árbol.

Parámetros

<i>data</i>	Elemento a buscar.
-------------	--------------------

Devuelve

true si existe, false en caso contrario.

3.1.3.15. searchRange()

```
template<typename T>
int BPlusTree< T >::searchRange (
    T start,
    T end,
    T * result_data,
    int arr_length)
```

Busca todos los elementos en el rango [start, end].

Parámetros

<i>start</i>	Límite inferior del rango.
<i>end</i>	Límite superior del rango.
<i>result_data</i>	Arreglo donde se almacenan los resultados.
<i>arr_length</i>	Longitud máxima del arreglo de resultados.

Devuelve

Número de elementos encontrados.

La documentación de esta clase está generada del siguiente archivo:

- [src/structures/BPlusTree.hpp](#)

3.2. Referencia de la estructura Column

Representa una columna de la tabla, con nombre y tipo de dato.

```
#include <Table.hpp>
```

Atributos públicos

- `std::string name`
Nombre de la columna.
- `DataType type`
Tipo de dato de la columna.

3.2.1. Descripción detallada

Representa una columna de la tabla, con nombre y tipo de dato.

La documentación de esta estructura está generada del siguiente archivo:

- `src/system/`[Table.hpp](#)

3.3. Referencia de la plantilla de la clase `HashMap< K, V >`

Implementación genérica de un mapa hash con direccionamiento abierto y redimensionamiento.

```
#include <HashMap.hpp>
```

Métodos públicos

- `HashMap (int cap)`
Constructor de [HashMap](#).
- `~HashMap ()`
Destructor de [HashMap](#).
- `long long * _getPows ()`
Calcula y retorna el arreglo de potencias para el hash.
- `double _getLoadFactor ()`
Calcula el factor de carga actual de la tabla.
- `std::string _toStringGeneric (const K &val)`
Convierte un valor genérico a string para el hash.
- `int _hashCode (K key)`
Calcula el código hash para una clave.
- `int _wrap (long long key, int size)`
Ajusta un valor hash al rango de la tabla.
- `void _insert (K key, V value, HashNode< K, V > **arr)`
Inserta un par clave-valor en un arreglo de nodos.
- `void _grow ()`
Duplica la capacidad de la tabla y reubica los elementos.

- void **insert** (K key, V value)
Inserta un par clave-valor en el mapa.
- std::optional< V > **get** (K key)
Obtiene el valor asociado a una clave.
- std::optional< V > **remove** (K key)
Elimina un par clave-valor del mapa.
- bool **containsKey** (K key)
Verifica si una clave existe en el mapa.
- int **getSize** ()
Retorna el número de elementos almacenados.
- bool **isEmpty** ()
Verifica si el mapa está vacío.
- **HashNode**< K, V > ** **toList** ()
Retorna un arreglo con todos los nodos almacenados.
- void **display** ()
Muestra por consola todos los pares clave-valor almacenados.
- void **clear** ()
Elimina todos los elementos del mapa y libera la memoria.

Atributos privados

- **HashNode**< K, V > ** **table**
Tabla hash de punteros a nodos.
- int **capacity**
Capacidad actual de la tabla.
- int **size**
Número de elementos almacenados.
- long long * **pows**
Potencias precomputadas para el hash.

3.3.1. Descripción detallada

template<typename K, typename V>
class **HashMap**< K, V >

Implementación genérica de un mapa hash con direccionamiento abierto y redimensionamiento.

Permite almacenar, buscar, eliminar y mostrar pares clave-valor. Utiliza direccionamiento abierto con sondeo cuadrático para resolver colisiones y redimensiona automáticamente la tabla cuando la carga supera cierto umbral.

Parámetros de plantilla

<i>K</i>	Tipo de la clave.
<i>V</i>	Tipo del valor.

3.3.2. Documentación de constructores y destructores

3.3.2.1. HashMap()

```
template<typename K, typename V>
HashMap< K, V >::HashMap (
    int cap)
```

Constructor de **HashMap**.

Parámetros

<i>cap</i>	Capacidad inicial de la tabla hash.
------------	-------------------------------------

3.3.2.2. `~HashMap()`

```
template<typename K, typename V>
HashMap< K, V >::~~HashMap ()
```

Destructor de `HashMap`.

Libera la memoria utilizada.

3.3.3. Documentación de funciones miembro**3.3.3.1. `_getLoadFactor()`**

```
template<typename K, typename V>
double HashMap< K, V >::_getLoadFactor ()
```

Calcula el factor de carga actual de la tabla.

Devuelve

Factor de carga (size/capacity).

3.3.3.2. `_getPows()`

```
template<typename K, typename V>
long long * HashMap< K, V >::_getPows ()
```

Calcula y retorna el arreglo de potencias para el hash.

Devuelve

Puntero al arreglo de potencias.

3.3.3.3. `_hashCode()`

```
template<typename K, typename V>
int HashMap< K, V >::_hashCode (
    K key)
```

Calcula el código hash para una clave.

Parámetros

<i>key</i>	Clave a hashear.
------------	------------------

Devuelve

Índice hash calculado.

3.3.3.4. `_insert()`

```
template<typename K, typename V>
void HashMap< K, V >::_insert (
    K key,
    V value,
    HashNode< K, V > ** arr)
```

Inserta un par clave-valor en un arreglo de nodos.

Parámetros

<i>key</i>	Clave a insertar.
<i>value</i>	Valor a insertar.
<i>arr</i>	Arreglo de nodos donde insertar.

3.3.3.5. `_toStringGeneric()`

```
template<typename K, typename V>
std::string HashMap< K, V >::_toStringGeneric (
    const K & val)
```

Convierte un valor genérico a string para el hash.

Parámetros

<i>val</i>	Valor a convertir.
------------	--------------------

Devuelve

Representación en string del valor.

3.3.3.6. `_wrap()`

```
template<typename K, typename V>
int HashMap< K, V >::_wrap (
    long long key,
    int size)
```

Ajusta un valor hash al rango de la tabla.

Parámetros

<i>key</i>	Valor hash.
<i>size</i>	Tamaño de la tabla.

Devuelve

Valor ajustado al rango [0, size).

3.3.3.7. `containsKey()`

```
template<typename K, typename V>
bool HashMap< K, V >::containsKey (
    K key)
```

Verifica si una clave existe en el mapa.

Parámetros

<i>key</i>	Clave a buscar.
------------	-----------------

Devuelve

true si existe, false en caso contrario.

3.3.3.8. `get()`

```
template<typename K, typename V>
std::optional< V > HashMap< K, V >::get (
    K key)
```

Obtiene el valor asociado a una clave.

Parámetros

<i>key</i>	Clave a buscar.
------------	-----------------

Devuelve

Valor asociado si existe, `std::nullopt` en caso contrario.

3.3.3.9. `getSize()`

```
template<typename K, typename V>
int HashMap< K, V >::getSize ()
```

Retorna el número de elementos almacenados.

Devuelve

Número de elementos.

3.3.3.10. insert()

```
template<typename K, typename V>
void HashMap< K, V >::insert (
    K key,
    V value)
```

Inserta un par clave-valor en el mapa.

Parámetros

<i>key</i>	Clave a insertar.
<i>value</i>	Valor a insertar.

3.3.3.11. isEmpty()

```
template<typename K, typename V>
bool HashMap< K, V >::isEmpty ()
```

Verifica si el mapa está vacío.

Devuelve

true si está vacío, false en caso contrario.

3.3.3.12. remove()

```
template<typename K, typename V>
std::optional< V > HashMap< K, V >::remove (
    K key)
```

Elimina un par clave-valor del mapa.

Parámetros

<i>key</i>	Clave a eliminar.
------------	-------------------

Devuelve

Valor eliminado si existía, std::nullopt en caso contrario.

3.3.3.13. toList()

```
template<typename K, typename V>
HashNode< K, V > ** HashMap< K, V >::toList ()
```

Retorna un arreglo con todos los nodos almacenados.

Devuelve

Puntero al arreglo de nodos.

La documentación de esta clase está generada del siguiente archivo:

- [src/structures/HashMap.hpp](#)

3.4. Referencia de la plantilla de la clase `HashNode< K, V >`

Nodo que almacena un par clave-valor en la tabla hash.

```
#include <HashMap.hpp>
```

Métodos públicos

- `HashNode` (`K key`, `V value`)
Constructor de `HashNode`.

Atributos públicos

- `V value`
Valor almacenado en el nodo.
- `K key`
Clave asociada al valor.

3.4.1. Descripción detallada

```
template<typename K, typename V>  
class HashNode< K, V >
```

Nodo que almacena un par clave-valor en la tabla hash.

Parámetros de plantilla

<i>K</i>	Tipo de la clave.
<i>V</i>	Tipo del valor.

3.4.2. Documentación de constructores y destructores

3.4.2.1. `HashNode()`

```
template<typename K, typename V>  
HashNode< K, V >::HashNode (  
    K key,  
    V value)
```

Constructor de `HashNode`.

Parámetros

<i>key</i>	Clave del nodo.
<i>value</i>	Valor asociado a la clave.

La documentación de esta clase está generada del siguiente archivo:

- `src/structures/HashMap.hpp`

3.5. Referencia de la estructura IndexEntry

Entrada de índice para estructuras de índice (B+ o invertido).

```
#include <IndexEntry.hpp>
```

Métodos públicos

- bool `operator<` (const `IndexEntry` &other) const
Operador de comparación menor que.
- bool `operator>` (const `IndexEntry` &other) const
Operador de comparación mayor que.
- bool `operator<=` (const `IndexEntry` &other) const
Operador de comparación menor o igual que.
- bool `operator>=` (const `IndexEntry` &other) const
Operador de comparación mayor o igual que.
- bool `operator==` (const `IndexEntry` &other) const
Operador de comparación de igualdad.

Atributos públicos

- `Cell key`
Clave del índice (valor de celda).
- int `rowId`
Identificador de la fila.

3.5.1. Descripción detallada

Entrada de índice para estructuras de índice (B+ o invertido).

Contiene la clave (valor de celda) y el identificador de fila correspondiente.

3.5.2. Documentación de funciones miembro

3.5.2.1. `operator<()`

```
bool IndexEntry::operator< (  
    const IndexEntry & other) const
```

Operador de comparación menor que.

Compara dos `IndexEntry` basándose en su clave.

3.5.2.2. `operator<=()`

```
bool IndexEntry::operator<= (  
    const IndexEntry & other) const
```

Operador de comparación menor o igual que.

Compara dos `IndexEntry` basándose en su clave.

3.5.2.3. operator==()

```
bool IndexEntry::operator== (
    const IndexEntry & other) const
```

Operador de comparación de igualdad.

Compara dos [IndexEntry](#) basándose en su clave y rowId.

3.5.2.4. operator>()

```
bool IndexEntry::operator> (
    const IndexEntry & other) const
```

Operador de comparación mayor que.

Compara dos [IndexEntry](#) basándose en su clave.

3.5.2.5. operator>=()

```
bool IndexEntry::operator>= (
    const IndexEntry & other) const
```

Operador de comparación mayor o igual que.

Compara dos [IndexEntry](#) basándose en su clave.

La documentación de esta estructura está generada de los siguientes archivos:

- [src/system/IndexEntry.hpp](#)
- [src/system/IndexEntry.cpp](#)

3.6. Referencia de la clase InvertedIndex

Índice invertido basado en [HashMap](#) para asociar claves con listas de identificadores.

```
#include <InvertedIndex.hpp>
```

Métodos públicos

- [InvertedIndex](#) (int initialCapacity=101)
Constructor de [InvertedIndex](#).
- [~InvertedIndex](#) ()
Destructor de [InvertedIndex](#).
- void [add](#) (const std::string &key, int rowId)
Agrega una asociación entre una clave y un identificador de fila.
- std::vector< int > [get](#) (const std::string &key)
Obtiene la lista de identificadores asociados a una clave.
- void [remove](#) (const std::string &key, int rowId)
Elimina la asociación de un identificador de fila para una clave dada.
- void [clear](#) ()
Elimina todas las asociaciones del índice invertido.

Atributos privados

- `HashMap< std::string, std::vector< int > > map_`
Mapa hash que almacena las asociaciones clave-lista de identificadores.

3.6.1. Descripción detallada

Índice invertido basado en `HashMap` para asociar claves con listas de identificadores.

Permite agregar, obtener, eliminar y limpiar asociaciones entre claves y listas de enteros.

3.6.2. Documentación de constructores y destructores

3.6.2.1. `InvertedIndex()`

```
InvertedIndex::InvertedIndex (
    int initialCapacity = 101)
```

Constructor de `InvertedIndex`.

Parámetros

<i>initialCapacity</i>	Capacidad inicial del mapa hash subyacente (por defecto 101).
------------------------	---

3.6.2.2. `~InvertedIndex()`

```
InvertedIndex::~~InvertedIndex ()
```

Destructor de `InvertedIndex`.

Libera la memoria utilizada.

3.6.3. Documentación de funciones miembro

3.6.3.1. `add()`

```
void InvertedIndex::add (
    const std::string & key,
    int rowId)
```

Agrega una asociación entre una clave y un identificador de fila.

Parámetros

<i>key</i>	Clave a asociar.
<i>rowId</i>	Identificador de fila a agregar.

3.6.3.2. `get()`

```
std::vector< int > InvertedIndex::get (
    const std::string & key)
```

Obtiene la lista de identificadores asociados a una clave.

Parámetros

<i>key</i>	Clave a buscar.
------------	-----------------

Devuelve

Vector de identificadores asociados. Si la clave no existe, retorna un vector vacío.

3.6.3.3. remove()

```
void InvertedIndex::remove (
    const std::string & key,
    int rowId)
```

Elimina la asociación de un identificador de fila para una clave dada.

Parámetros

<i>key</i>	Clave de la que se eliminará el identificador.
<i>rowId</i>	Identificador de fila a eliminar.

La documentación de esta clase está generada de los siguientes archivos:

- [src/structures/InvertedIndex.hpp](#)
- [src/structures/InvertedIndex.cpp](#)

3.7. Referencia de la plantilla de la estructura Node< T >

Representa un nodo en el árbol B+.

```
#include <BPlusTree.hpp>
```

Métodos públicos

- [Node](#) (int *_degree*)
Constructor del nodo.

Atributos públicos

- bool **is_leaf**
Indica si el nodo es una hoja.
- int **degree**
Grado del nodo, que determina cuántos hijos puede tener.
- int **size**
Número actual de elementos en el nodo.
- T * **item**
Arreglo de elementos almacenados en el nodo.
- [Node](#)< T > ** **children**
Arreglo de punteros a los hijos del nodo.
- [Node](#)< T > * **parent**
Puntero al nodo padre.

3.7.1. Descripción detallada

```
template<typename T>
struct Node< T >
```

Representa un nodo en el árbol B+.

Cada nodo puede ser una hoja o un nodo interno. Los nodos internos contienen claves y punteros a sus hijos, mientras que las hojas contienen los datos finales del árbol.

3.7.2. Documentación de constructores y destructores

3.7.2.1. Node()

```
template<typename T>
Node< T >::Node (
    int _degree)
```

Constructor del nodo.

Constructor de un nodo del árbol B+.

Parámetros

<code>_degree</code>	Grado del nodo, que determina cuántos hijos puede tener.
----------------------	--

Inicializa un nodo con el grado especificado. El nodo es conectado al árbol B+ en la función de inserción.

Parámetros

<code>_degree</code>	El grado del nodo, que determina cuántos hijos puede tener este nodo.
----------------------	---

La documentación de esta estructura está generada del siguiente archivo:

- [src/structures/BPlusTree.hpp](#)

3.8. Referencia de la clase Table

Representa una tabla de datos con soporte para índices y operaciones básicas.

```
#include <Table.hpp>
```

Métodos públicos

- **Table** (const std::vector< std::pair< std::string, **DataType** > > &cols, int hashCapacity=101)
Constructor de Table.
- **~Table** ()
Destructor de Table.
- void **insertRow** (const std::vector< Cell > &row)
Inserta una nueva fila en la tabla.
- void **printSchema** () const
Imprime el esquema (columnas y tipos) de la tabla.
- void **printAllRows** () const
Imprime todas las filas almacenadas en la tabla.
- Cell **getCell** (int rowIndex, const std::string &colName)
Obtiene el valor de una celda dado el índice de fila y el nombre de columna.
- void **createBPlusTreeIndex** (const std::string &colName, int degree)
Crea un índice B+ sobre una columna específica.
- void **createInvertedIndex** (const std::string &colName)
Crea un índice invertido sobre una columna específica.
- std::vector< int > **searchByIndex** (const std::string &colName, const Cell &key)
Busca filas por valor de clave usando el índice correspondiente.

Métodos privados

- int **columnIndex** (const std::string &colName)
Obtiene el índice de una columna dado su nombre.

Métodos privados estáticos

- static bool **cellMatchesType** (const Cell &cell, **DataType** dt)
Verifica si una celda coincide con un tipo de dato.
- static std::string **dataTypeToString** (**DataType** dt)
Convierte un tipo de dato a su representación en string.

Atributos privados

- std::vector< **Column** > **schema_**
Esquema de columnas de la tabla.
- **HashMap**< std::string, int > **nameToIndex_**
Mapa de nombre de columna a índice.
- std::vector< std::vector< Cell > > **data_**
Datos almacenados en la tabla.
- **HashMap**< std::string, **BPlusTree**< **IndexEntry** > * > **bptIndices_**
Índices B+ por columna.
- **HashMap**< std::string, **InvertedIndex** * > **invIndices_**
Índices invertidos por columna.

3.8.1. Descripción detallada

Representa una tabla de datos con soporte para índices y operaciones básicas.

Permite definir un esquema de columnas, insertar filas, crear índices B+ e invertidos, y realizar búsquedas eficientes por índice.

3.8.2. Documentación de constructores y destructores

3.8.2.1. Table()

```
Table::Table (
    const std::vector< std::pair< std::string, DataType > > & cols,
    int hashCapacity = 101)
```

Constructor de [Table](#).

Parámetros

<i>cols</i>	Vector de pares (nombre, tipo) que define el esquema de la tabla.
<i>hashCapacity</i>	Capacidad inicial para los mapas hash internos (por defecto 101).

3.8.2.2. ~Table()

```
Table::~~Table ()
```

Destructor de [Table](#).

Libera la memoria utilizada por los índices.

3.8.3. Documentación de funciones miembro

3.8.3.1. cellMatchesType()

```
bool Table::cellMatchesType (
    const Cell & cell,
    DataType dt) [static], [private]
```

Verifica si una celda coincide con un tipo de dato.

Verifica si una celda coincide con un tipo de dato específico.

Parámetros

<i>cell</i>	Celda a verificar.
<i>dt</i>	Tipo de dato esperado.

Devuelve

true si coincide, false en caso contrario.

3.8.3.2. columnIndex()

```
int Table::columnIndex (
    const std::string & colName) [private]
```

Obtiene el índice de una columna dado su nombre.

Parámetros

<i>colName</i>	Nombre de la columna.
----------------	-----------------------

Devuelve

Índice de la columna en el esquema.

3.8.3.3. createBPlusTreeIndex()

```
void Table::createBPlusTreeIndex (  
    const std::string & colName,  
    int degree)
```

Crea un índice B+ sobre una columna específica.

Parámetros

<i>colName</i>	Nombre de la columna a indexar.
<i>degree</i>	Grado del árbol B+.

3.8.3.4. createInvertedIndex()

```
void Table::createInvertedIndex (  
    const std::string & colName)
```

Crea un índice invertido sobre una columna específica.

Parámetros

<i>colName</i>	Nombre de la columna a indexar.
----------------	---------------------------------

3.8.3.5. dataTypeToString()

```
std::string Table::dataTypeToString (  
    DataType dt) [static], [private]
```

Convierte un tipo de dato a su representación en string.

Parámetros

<i>dt</i>	Tipo de dato.
-----------	---------------

Devuelve

String representando el tipo de dato.

3.8.3.6. getCell()

```
Cell Table::getCell (  
    int rowIndex,  
    const std::string & colName)
```

Obtiene el valor de una celda dado el índice de fila y el nombre de columna.

Parámetros

<i>rowIndex</i>	Índice de la fila.
<i>colName</i>	Nombre de la columna.

Devuelve

Valor de la celda correspondiente.

3.8.3.7. insertRow()

```
void Table::insertRow (  
    const std::vector< Cell > & row)
```

Inserta una nueva fila en la tabla.

Parámetros

<i>row</i>	Vector de celdas que representa la fila a insertar.
------------	---

3.8.3.8. searchByIndex()

```
std::vector< int > Table::searchByIndex (  
    const std::string & colName,  
    const Cell & key)
```

Busca filas por valor de clave usando el índice correspondiente.

Parámetros

<i>colName</i>	Nombre de la columna indexada.
<i>key</i>	Valor de la clave a buscar.

Devuelve

Vector de identificadores de filas que coinciden con la clave.

La documentación de esta clase está generada de los siguientes archivos:

- [src/system/Table.hpp](#)
- [src/system/Table.cpp](#)

Capítulo 4

Documentación de archivos

4.1. Referencia del archivo src/structures/BPlusTree.hpp

Implementación de un árbol B+ genérico.

```
#include <iostream>
```

Clases

- struct `Node< T >`
Representa un nodo en el árbol B+.
- class `BPlusTree< T >`
Implementación genérica de un árbol B+.

4.1.1. Descripción detallada

Implementación de un árbol B+ genérico.

Este árbol B+ permite insertar, buscar y eliminar elementos de tipo T.

4.2. BPlusTree.hpp

[Ir a la documentación de este archivo.](#)

```
00001 #include <iostream>
00002
00007
00015 template <typename T> struct Node {
00016     bool is_leaf;
00017     int degree;
00018     int size;
00019     T *item;
00020     Node<T> **children;
00021     Node<T> *parent;
00022
00023 public:
00028     Node(int _degree);
00029 };
00030
```

```

00042 template <typename T> class BPlusTree {
00043     Node<T> *root;
00044     int degree;
00045
00046 public:
00051     BPlusTree(int _degree);
00052     ~BPlusTree();
00053
00065     Node<T> *_findKey(Node<T> *node, T key);
00066
00073     Node<T> *_findRange(Node<T> *node, T key);
00074
00082     int _findIndex(T *arr, T data, int len);
00083
00092     Node<T> **_innerInsert(Node<T> **child_arr, Node<T> *child, int len,
00093                           int index);
00094
00102     T *_insertItem(T *arr, T data, int len);
00103
00111     Node<T> *_insertInChild(Node<T> *node, T data, Node<T> *child);
00112
00119     void _insertInParent(Node<T> *par, Node<T> *child, T data);
00120
00127     void _removeFromParent(Node<T> *node, int index, Node<T> *par);
00128
00133     void _print(Node<T> *cursor);
00134
00139     Node<T> *getRoot();
00140
00149     int searchRange(T start, T end, T *result_data, int arr_length);
00150
00156     bool search(T data);
00157
00162     void insert(T data);
00163
00168     void remove(T data);
00169
00174     void clear(Node<T> *cursor);
00175
00179     void print();
00180 };
00181
00189 template <typename T> Node<T>::Node(int _degree) {
00190     this->is_leaf = false;
00191     this->degree = _degree;
00192     this->size = 0;
00193
00194     T *_item = new T[degree - 1]();
00195     this->item = _item;
00196
00197     Node<T> **_children = new Node<T> *[degree];
00198     for (int i = 0; i < degree; i++) {
00199         _children[i] = nullptr;
00200     }
00201     this->children = _children;
00202
00203     this->parent = nullptr;
00204 }
00205
00206 template <typename T> BPlusTree<T>::BPlusTree(int _degree) {
00207     this->root = nullptr;
00208     this->degree = _degree;
00209 }
00210
00211 template <typename T> BPlusTree<T>::~BPlusTree() { clear(this->root); }
00212
00213 template <typename T> Node<T> *BPlusTree<T>::getRoot() { return this->root; }
00214
00215 template <typename T> Node<T> *BPlusTree<T>::_findKey(Node<T> *node, T key) {
00216     if (node == nullptr) {
00217         return nullptr;
00218     } else {
00219         Node<T> *cursor = node;
00220
00221         while (!cursor->is_leaf) {
00222             for (int i = 0; i < cursor->size; i++) {
00223                 if (key < cursor->item[i]) {
00224                     cursor = cursor->children[i];
00225                     break;
00226                 }
00227             }
00228             if (i == (cursor->size) - 1) {
00229                 cursor = cursor->children[i + 1];
00230                 break;
00231             }
00232         }
00233     }

```

```

00233     }
00234
00235     for (int i = 0; i < cursor->size; i++) {
00236         if (cursor->item[i] == key) {
00237             return cursor;
00238         }
00239     }
00240
00241     return nullptr;
00242 }
00243 }
00244
00245 template <typename T> Node<T> *BPlusTree<T>::_findRange(Node<T> *node, T key) {
00246     if (node == nullptr) {
00247         return nullptr;
00248     } else {
00249         Node<T> *cursor = node;
00250
00251         while (!cursor->is_leaf) {
00252             for (int i = 0; i < cursor->size; i++) {
00253                 if (key < cursor->item[i]) {
00254
00255                     cursor = cursor->children[i];
00256                     break;
00257                 }
00258                 if (i == (cursor->size) - 1) {
00259                     cursor = cursor->children[i + 1];
00260                     break;
00261                 }
00262             }
00263         }
00264         return cursor;
00265     }
00266 }
00267
00268 template <typename T>
00269 int BPlusTree<T>::searchRange(T start, T end, T *result_data, int arr_length) {
00270     int index = 0;
00271
00272     Node<T> *start_node = _findRange(this->root, start);
00273     Node<T> *cursor = start_node;
00274     T temp = cursor->item[0];
00275
00276     while (temp <= end) {
00277         if (cursor == nullptr) {
00278             break;
00279         }
00280         for (int i = 0; i < cursor->size; i++) {
00281             temp = cursor->item[i];
00282             if ((temp >= start) && (temp <= end)) {
00283                 result_data[index] = temp;
00284                 index++;
00285             }
00286         }
00287         cursor = cursor->children[cursor->size];
00288     }
00289     return index;
00290 }
00291
00292 template <typename T> bool BPlusTree<T>::search(T data) {
00293     return _findKey(this->root, data) != nullptr;
00294 }
00295
00296 template <typename T> int BPlusTree<T>::_findIndex(T *arr, T data, int len) {
00297     int index = 0;
00298     for (int i = 0; i < len; i++) {
00299         if (data < arr[i]) {
00300             index = i;
00301             break;
00302         }
00303         if (i == len - 1) {
00304             index = len;
00305             break;
00306         }
00307     }
00308     return index;
00309 }
00310
00311 template <typename T> T *BPlusTree<T>::_insertItem(T *arr, T data, int len) {
00312     int index = 0;
00313     for (int i = 0; i < len; i++) {
00314         if (data < arr[i]) {
00315             index = i;
00316             break;
00317         }
00318         if (i == len - 1) {
00319             index = len;

```

```

00320         break;
00321     }
00322 }
00323
00324 for (int i = len; i > index; i--) {
00325     arr[i] = arr[i - 1];
00326 }
00327
00328 arr[index] = data;
00329
00330 return arr;
00331 }
00332
00333 template <typename T>
00334 Node<T> **BPlusTree<T>::_innerInsert(Node<T> **child_arr, Node<T> *child,
00335                                     int len, int index) {
00336     for (int i = len; i > index; i--) {
00337         child_arr[i] = child_arr[i - 1];
00338     }
00339     child_arr[index] = child;
00340     return child_arr;
00341 }
00342
00343 template <typename T>
00344 Node<T> *BPlusTree<T>::_insertInChild(Node<T> *node, T data, Node<T> *child) {
00345     int item_index = 0;
00346     int child_index = 0;
00347     for (int i = 0; i < node->size; i++) {
00348         if (data < node->item[i]) {
00349             item_index = i;
00350             child_index = i + 1;
00351             break;
00352         }
00353         if (i == node->size - 1) {
00354             item_index = node->size;
00355             child_index = node->size + 1;
00356             break;
00357         }
00358     }
00359     for (int i = node->size; i > item_index; i--) {
00360         node->item[i] = node->item[i - 1];
00361     }
00362     for (int i = node->size + 1; i > child_index; i--) {
00363         node->children[i] = node->children[i - 1];
00364     }
00365
00366     node->item[item_index] = data;
00367     node->children[child_index] = child;
00368
00369     return node;
00370 }
00371
00372 template <typename T>
00373 void BPlusTree<T>::_insertInParent(Node<T> *par, Node<T> *child, T data) {
00374
00375     Node<T> *cursor = par;
00376     if (cursor->size < this->degree - 1) {
00377
00378         cursor = _insertInChild(cursor, data, child);
00379         cursor->size++;
00380     } else {
00381
00382         auto *Newnode = new Node<T>(this->degree);
00383         Newnode->parent = cursor->parent;
00384
00385         T *item_copy = new T[cursor->size + 1];
00386         for (int i = 0; i < cursor->size; i++) {
00387             item_copy[i] = cursor->item[i];
00388         }
00389         item_copy = _insertItem(item_copy, data, cursor->size);
00390
00391         auto **child_copy = new Node<T> *[cursor->size + 2];
00392         for (int i = 0; i < cursor->size + 1; i++) {
00393             child_copy[i] = cursor->children[i];
00394         }
00395         child_copy[cursor->size + 1] = nullptr;
00396         child_copy = _innerInsert(child_copy, child, cursor->size + 1,
00397                                 _findIndex(item_copy, data, cursor->size + 1));
00398
00399         cursor->size = (this->degree) / 2;
00400         if ((this->degree) % 2 == 0) {
00401             Newnode->size = (this->degree) / 2 - 1;
00402         } else {
00403             Newnode->size = (this->degree) / 2;
00404         }
00405
00406         for (int i = 0; i < cursor->size; i++) {

```

```

00407     cursor->item[i] = item_copy[i];
00408     cursor->children[i] = child_copy[i];
00409 }
00410 cursor->children[cursor->size] = child_copy[cursor->size];
00411
00412 for (int i = 0; i < Newnode->size; i++) {
00413     Newnode->item[i] = item_copy[cursor->size + i + 1];
00414     Newnode->children[i] = child_copy[cursor->size + i + 1];
00415     Newnode->children[i]->parent = Newnode;
00416 }
00417 Newnode->children[Newnode->size] =
00418     child_copy[cursor->size + Newnode->size + 1];
00419 Newnode->children[Newnode->size]->parent = Newnode;
00420
00421 T paritem = item_copy[this->degree / 2];
00422
00423 delete[] item_copy;
00424 delete[] child_copy;
00425
00426 if (cursor->parent == nullptr) {
00427     auto *Newparent = new Node<T>(this->degree);
00428     cursor->parent = Newparent;
00429     Newnode->parent = Newparent;
00430
00431     Newparent->item[0] = paritem;
00432     Newparent->size++;
00433
00434     Newparent->children[0] = cursor;
00435     Newparent->children[1] = Newnode;
00436
00437     this->root = Newparent;
00438 } else {
00439     _insertInParent(cursor->parent, Newnode, paritem);
00440 }
00441 }
00442 }
00443 }
00444
00445 template <typename T> void BPlusTree<T>::insert(T data) {
00446     if (this->root == nullptr) {
00447         this->root = new Node<T>(this->degree);
00448         this->root->is_leaf = true;
00449         this->root->item[0] = data;
00450         this->root->size = 1;
00451     } else {
00452         Node<T> *cursor = this->root;
00453
00454         cursor = _findRange(cursor, data);
00455
00456         if (cursor->size < (this->degree - 1)) {
00457
00458             cursor->item = _insertItem(cursor->item, data, cursor->size);
00459             cursor->size++;
00460
00461             cursor->children[cursor->size] = cursor->children[cursor->size - 1];
00462             cursor->children[cursor->size - 1] = nullptr;
00463         } else {
00464
00465             auto *Newnode = new Node<T>(this->degree);
00466             Newnode->is_leaf = true;
00467             Newnode->parent = cursor->parent;
00468
00469             T *item_copy = new T[cursor->size + 1];
00470             for (int i = 0; i < cursor->size; i++) {
00471                 item_copy[i] = cursor->item[i];
00472             }
00473
00474             item_copy = _insertItem(item_copy, data, cursor->size);
00475
00476             cursor->size = (this->degree) / 2;
00477             if ((this->degree) % 2 == 0) {
00478                 Newnode->size = (this->degree) / 2;
00479             } else {
00480                 Newnode->size = (this->degree) / 2 + 1;
00481             }
00482
00483             for (int i = 0; i < cursor->size; i++) {
00484                 cursor->item[i] = item_copy[i];
00485             }
00486             for (int i = 0; i < Newnode->size; i++) {
00487                 Newnode->item[i] = item_copy[cursor->size + i];
00488             }
00489
00490             cursor->children[cursor->size] = Newnode;
00491             Newnode->children[Newnode->size] = cursor->children[this->degree - 1];
00492             cursor->children[this->degree - 1] = nullptr;
00493

```

```

00494     delete[] item_copy;
00495
00496     T paritem = Newnode->item[0];
00497
00498     if (cursor->parent == nullptr) {
00499         auto *Newparent = new Node<T>(this->degree);
00500         cursor->parent = Newparent;
00501         Newnode->parent = Newparent;
00502
00503         Newparent->item[0] = paritem;
00504         Newparent->size++;
00505
00506         Newparent->children[0] = cursor;
00507         Newparent->children[1] = Newnode;
00508
00509         this->root = Newparent;
00510     } else {
00511         _insertInParent(cursor->parent, Newnode, paritem);
00512     }
00513 }
00514 }
00515 }
00516
00517 template <typename T> void BPlusTree<T>::remove(T data) {
00518
00519     Node<T> *cursor = this->root;
00520
00521     cursor = _findRange(cursor, data);
00522
00523     int sib_index = -1;
00524     for (int i = 0; i < cursor->parent->size + 1; i++) {
00525         if (cursor == cursor->parent->children[i]) {
00526             sib_index = i;
00527         }
00528     }
00529     int left = sib_index - 1;
00530     int right = sib_index + 1;
00531
00532     int del_index = -1;
00533     for (int i = 0; i < cursor->size; i++) {
00534         if (cursor->item[i] == data) {
00535             del_index = i;
00536             break;
00537         }
00538     }
00539
00540     if (del_index == -1) {
00541         return;
00542     }
00543
00544     for (int i = del_index; i < cursor->size - 1; i++) {
00545         cursor->item[i] = cursor->item[i + 1];
00546     }
00547     cursor->item[cursor->size - 1] = 0;
00548     cursor->size--;
00549
00550     if (cursor == this->root && cursor->size == 0) {
00551         clear(this->root);
00552         this->root = nullptr;
00553         return;
00554     }
00555     cursor->children[cursor->size] = cursor->children[cursor->size + 1];
00556     cursor->children[cursor->size + 1] = nullptr;
00557
00558     if (cursor == this->root) {
00559         return;
00560     }
00561     if (cursor->size < degree / 2) {
00562
00563         if (left >= 0) {
00564             Node<T> *leftsibling = cursor->parent->children[left];
00565
00566             if (leftsibling->size > degree / 2) {
00567                 T *temp = new T[cursor->size + 1];
00568
00569                 for (int i = 0; i < cursor->size; i++) {
00570                     temp[i] = cursor->item[i];
00571                 }
00572
00573                 _insertItem(temp, leftsibling->item[leftsibling->size - 1],
00574                             cursor->size);
00575                 for (int i = 0; i < cursor->size + 1; i++) {
00576                     cursor->item[i] = temp[i];
00577                 }
00578                 cursor->size++;
00579                 delete[] temp;
00580

```



```

00581         cursor->children[cursor->size] = cursor->children[cursor->size - 1];
00582         cursor->children[cursor->size - 1] = nullptr;
00583
00584         leftsibling->item[leftsibling->size - 1] = 0;
00585         leftsibling->size--;
00586         leftsibling->children[leftsibling->size] =
00587             leftsibling->children[leftsibling->size + 1];
00588         leftsibling->children[leftsibling->size + 1] = nullptr;
00589
00590         cursor->parent->item[left] = cursor->item[0];
00591
00592         return;
00593     }
00594 }
00595 if (right <= cursor->parent->size) {
00596     Node<T> *rightsibling = cursor->parent->children[right];
00597
00598     if (rightsibling->size > degree / 2) {
00599         T *temp = new T[cursor->size + 1];
00600
00601         for (int i = 0; i < cursor->size; i++) {
00602             temp[i] = cursor->item[i];
00603         }
00604
00605         _insertItem(temp, rightsibling->item[0], cursor->size);
00606         for (int i = 0; i < cursor->size + 1; i++) {
00607             cursor->item[i] = temp[i];
00608         }
00609         cursor->size++;
00610         delete[] temp;
00611
00612         cursor->children[cursor->size] = cursor->children[cursor->size - 1];
00613         cursor->children[cursor->size - 1] = nullptr;
00614
00615         for (int i = 0; i < rightsibling->size - 1; i++) {
00616             rightsibling->item[i] = rightsibling->item[i + 1];
00617         }
00618         rightsibling->item[rightsibling->size - 1] = 0;
00619         rightsibling->size--;
00620         rightsibling->children[rightsibling->size] =
00621             rightsibling->children[rightsibling->size + 1];
00622         rightsibling->children[rightsibling->size + 1] = nullptr;
00623
00624         cursor->parent->item[right - 1] = rightsibling->item[0];
00625
00626         return;
00627     }
00628 }
00629
00630 if (left >= 0) {
00631     Node<T> *leftsibling = cursor->parent->children[left];
00632
00633     for (int i = 0; i < cursor->size; i++) {
00634         leftsibling->item[leftsibling->size + i] = cursor->item[i];
00635     }
00636
00637     leftsibling->children[leftsibling->size] = nullptr;
00638     leftsibling->size = leftsibling->size + cursor->size;
00639     leftsibling->children[leftsibling->size] = cursor->children[cursor->size];
00640
00641     _removeFromParent(cursor, left, cursor->parent);
00642     for (int i = 0; i < cursor->size; i++) {
00643         cursor->item[i] = 0;
00644         cursor->children[i] = nullptr;
00645     }
00646     cursor->children[cursor->size] = nullptr;
00647
00648     delete[] cursor->item;
00649     delete[] cursor->children;
00650     delete cursor;
00651
00652     return;
00653 }
00654 if (right <= cursor->parent->size) {
00655     Node<T> *rightsibling = cursor->parent->children[right];
00656
00657     for (int i = 0; i < rightsibling->size; i++) {
00658         cursor->item[i + cursor->size] = rightsibling->item[i];
00659     }
00660
00661     cursor->children[cursor->size] = nullptr;
00662     cursor->size = rightsibling->size + cursor->size;
00663     cursor->children[cursor->size] =
00664         rightsibling->children[rightsibling->size];
00665
00666     _removeFromParent(rightsibling, right - 1, cursor->parent);
00667 }

```

```

00668         for (int i = 0; i < rightsibling->size; i++) {
00669             rightsibling->item[i] = 0;
00670             rightsibling->children[i] = nullptr;
00671         }
00672         rightsibling->children[rightsibling->size] = nullptr;
00673
00674         delete[] rightsibling->item;
00675         delete[] rightsibling->children;
00676         delete rightsibling;
00677         return;
00678     }
00679
00680 } else {
00681     return;
00682 }
00683 }
00684
00685 template <typename T>
00686 void BPlusTree<T>::_removeFromParent(Node<T> *node, int index, Node<T> *par) {
00687     Node<T> *remover = node;
00688     Node<T> *cursor = par;
00689     T target = cursor->item[index];
00690
00691     if (cursor == this->root && cursor->size == 1) {
00692         if (remover == cursor->children[0]) {
00693             delete[] remover->item;
00694             delete[] remover->children;
00695             delete remover;
00696             this->root = cursor->children[1];
00697             delete[] cursor->item;
00698             delete[] cursor->children;
00699             delete cursor;
00700             return;
00701         }
00702         if (remover == cursor->children[1]) {
00703             delete[] remover->item;
00704             delete[] remover->children;
00705             delete remover;
00706             this->root = cursor->children[0];
00707             delete[] cursor->item;
00708             delete[] cursor->children;
00709             delete cursor;
00710             return;
00711         }
00712     }
00713
00714     for (int i = index; i < cursor->size - 1; i++) {
00715         cursor->item[i] = cursor->item[i + 1];
00716     }
00717     cursor->item[cursor->size - 1] = 0;
00718
00719     int rem_index = -1;
00720     for (int i = 0; i < cursor->size + 1; i++) {
00721         if (cursor->children[i] == remover) {
00722             rem_index = i;
00723         }
00724     }
00725     if (rem_index == -1) {
00726         return;
00727     }
00728     for (int i = rem_index; i < cursor->size; i++) {
00729         cursor->children[i] = cursor->children[i + 1];
00730     }
00731     cursor->children[cursor->size] = nullptr;
00732     cursor->size--;
00733
00734     if (cursor == this->root) {
00735         return;
00736     }
00737     if (cursor->size < degree / 2) {
00738
00739         int sib_index = -1;
00740         for (int i = 0; i < cursor->parent->size + 1; i++) {
00741             if (cursor == cursor->parent->children[i]) {
00742                 sib_index = i;
00743             }
00744         }
00745         int left = sib_index - 1;
00746         int right = sib_index + 1;
00747
00748         if (left >= 0) {
00749             Node<T> *leftsibling = cursor->parent->children[left];
00750
00751             if (leftsibling->size > degree / 2) {
00752                 T *temp = new T[cursor->size + 1];
00753
00754                 for (int i = 0; i < cursor->size; i++) {

```

```

00755         temp[i] = cursor->item[i];
00756     }
00757
00758     _insertItem(temp, cursor->parent->item[left], cursor->size);
00759     for (int i = 0; i < cursor->size + 1; i++) {
00760         cursor->item[i] = temp[i];
00761     }
00762     cursor->parent->item[left] = leftsibling->item[leftsibling->size - 1];
00763     delete[] temp;
00764
00765     Node<T> **child_temp = new Node<T> *[cursor->size + 2];
00766
00767     for (int i = 0; i < cursor->size + 1; i++) {
00768         child_temp[i] = cursor->children[i];
00769     }
00770
00771     _innerInsert(child_temp, leftsibling->children[leftsibling->size],
00772                 cursor->size, 0);
00773
00774     for (int i = 0; i < cursor->size + 2; i++) {
00775         cursor->children[i] = child_temp[i];
00776     }
00777     delete[] child_temp;
00778
00779     cursor->size++;
00780     leftsibling->size--;
00781     return;
00782 }
00783 }
00784
00785 if (right <= cursor->parent->size) {
00786     Node<T> *rightsibling = cursor->parent->children[right];
00787
00788     if (rightsibling->size > degree / 2) {
00789         T *temp = new T[cursor->size + 1];
00790
00791         for (int i = 0; i < cursor->size; i++) {
00792             temp[i] = cursor->item[i];
00793         }
00794
00795         _insertItem(temp, cursor->parent->item[sib_index], cursor->size);
00796         for (int i = 0; i < cursor->size + 1; i++) {
00797             cursor->item[i] = temp[i];
00798         }
00799         cursor->parent->item[sib_index] = rightsibling->item[0];
00800         delete[] temp;
00801
00802         cursor->children[cursor->size + 1] = rightsibling->children[0];
00803         for (int i = 0; i < rightsibling->size; i++) {
00804             rightsibling->children[i] = rightsibling->children[i + 1];
00805         }
00806         rightsibling->children[rightsibling->size] = nullptr;
00807
00808         cursor->size++;
00809         rightsibling->size--;
00810         return;
00811     }
00812 }
00813
00814 if (left >= 0) {
00815     Node<T> *leftsibling = cursor->parent->children[left];
00816
00817     leftsibling->item[leftsibling->size] = cursor->parent->item[left];
00818
00819     for (int i = 0; i < cursor->size; i++) {
00820         leftsibling->item[leftsibling->size + i + 1] = cursor->item[i];
00821     }
00822     for (int i = 0; i < cursor->size + 1; i++) {
00823         leftsibling->children[leftsibling->size + i + 1] = cursor->children[i];
00824         cursor->children[i]->parent = leftsibling;
00825     }
00826     for (int i = 0; i < cursor->size + 1; i++) {
00827         cursor->children[i] = nullptr;
00828     }
00829     leftsibling->size = leftsibling->size + cursor->size + 1;
00830
00831     _removeFromParent(cursor, left, cursor->parent);
00832     return;
00833 }
00834 if (right <= cursor->parent->size) {
00835     Node<T> *rightsibling = cursor->parent->children[right];
00836
00837     cursor->item[cursor->size] = cursor->parent->item[right - 1];
00838
00839     for (int i = 0; i < rightsibling->size; i++) {
00840         cursor->item[cursor->size + 1 + i] = rightsibling->item[i];
00841     }

```

```

00842     for (int i = 0; i < rightsibling->size + 1; i++) {
00843         cursor->children[cursor->size + i + 1] = rightsibling->children[i];
00844         rightsibling->children[i]->parent = rightsibling;
00845     }
00846     for (int i = 0; i < rightsibling->size + 1; i++) {
00847         rightsibling->children[i] = nullptr;
00848     }
00849
00850     rightsibling->size = rightsibling->size + cursor->size + 1;
00851
00852     _removeFromParent(rightsibling, right - 1, cursor->parent);
00853     return;
00854 }
00855 } else {
00856     return;
00857 }
00858 }
00859
00860 template <typename T> void BPlusTree<T>::clear(Node<T> *cursor) {
00861     if (cursor != nullptr) {
00862         if (!cursor->is_leaf) {
00863             for (int i = 0; i <= cursor->size; i++) {
00864                 clear(cursor->children[i]);
00865             }
00866         }
00867         delete[] cursor->item;
00868         delete[] cursor->children;
00869         delete cursor;
00870     }
00871 }
00872
00873 template <typename T> void BPlusTree<T>::print() { _print(this->root); }
00874
00875 template <typename T> void BPlusTree<T>::_print(Node<T> *cursor) {
00876     if (cursor != NULL) {
00877         for (int i = 0; i < cursor->size; ++i) {
00878             std::cout << cursor->item[i] << " ";
00879         }
00880         std::cout << "\n";
00881
00882         if (!cursor->is_leaf) {
00883             for (int i = 0; i < cursor->size + 1; ++i) {
00884                 _print(cursor->children[i]);
00885             }
00886         }
00887     }
00888 }
00889 }

```

4.3. Referencia del archivo src/structures/HashMap.hpp

Implementación de un mapa hash genérico con manejo de colisiones y redimensionamiento automático.

```

#include <iostream>
#include <optional>
#include <sstream>
#include <string>

```

Clases

- class `HashNode< K, V >`
Nodo que almacena un par clave-valor en la tabla hash.
- class `HashMap< K, V >`
Implementación genérica de un mapa hash con direccionamiento abierto y redimensionamiento.

4.3.1. Descripción detallada

Implementación de un mapa hash genérico con manejo de colisiones y redimensionamiento automático.

Este archivo define las clases `HashNode` y `HashMap`, que permiten almacenar pares clave-valor de tipo genérico, con operaciones de inserción, búsqueda, eliminación y utilidades adicionales.

4.4. HashMap.hpp

[Ir a la documentación de este archivo.](#)

```

00001
00010
00011 #include <iostream>
00012 #include <optional>
00013 #include <sstream>
00014 #include <string>
00015
00023 template <typename K, typename V> class HashNode {
00024 public:
00025     V value;
00026     K key;
00027
00033     HashNode(K key, V value);
00034 };
00035
00048 template <typename K, typename V> class HashMap {
00049     HashNode<K, V> **table;
00050     int capacity;
00051     int size;
00052     long long *pows;
00053
00054 public:
00059     HashMap(int cap);
00060
00064     ~HashMap();
00065
00070     long long *_getPows();
00071
00076     double _getLoadFactor();
00077
00083     std::string _toStringGeneric(const K &val);
00084
00090     int _hashCode(K key);
00091
00098     int _wrap(long long key, int size);
00099
00106     void _insert(K key, V value, HashNode<K, V> **arr);
00107
00111     void _grow();
00112
00118     void insert(K key, V value);
00119
00125     std::optional<V> get(K key);
00126
00132     std::optional<V> remove(K key);
00133
00139     bool containsKey(K key);
00140
00145     int getSize();
00146
00151     bool isEmpty();
00152
00157     HashNode<K, V> **toList();
00158
00162     void display();
00163
00167     void clear();
00168 };
00169
00170 template <typename K, typename V> HashNode<K, V>::HashNode(K key, V value) {
00171     this->value = value;
00172     this->key = key;
00173 }
00174
00175 template <typename K, typename V> HashMap<K, V>::HashMap(int cap) {
00176     capacity = cap;
00177     size = 0;
00178     table = new HashNode<K, V> *[capacity];
00179
00180     for (int i = 0; i < capacity; i++)
00181         table[i] = NULL;
00182
00183     pows = _getPows();
00184 }
00185
00186 template <typename K, typename V> HashMap<K, V>::~~HashMap() {
00187     for (int i = 0; i < capacity; i++) {
00188         if (table[i] != NULL) {
00189             delete table[i];
00190         }
00191     }
00192     delete[] table;

```

```

00193 }
00194
00195 template <typename K, typename V> void HashMap<K, V>::_grow() {
00196     HashNode<K, V> **oldTable = toList();
00197     capacity *= 2;
00198     HashNode<K, V> **newTable = new HashNode<K, V> *[capacity];
00199     for (int i = 0; i < capacity; i++) {
00200         HashNode<K, V> curr = *oldTable[i];
00201         _insert(curr.key, curr.value, newTable);
00202     }
00203 }
00204
00205 template <typename K, typename V>
00206 void HashMap<K, V>::_insert(K key, V value, HashNode<K, V> **arr) {
00207     HashNode<K, V> *temp = new HashNode<K, V>(key, value);
00208
00209     int hash = _hashCode(key);
00210
00211     int i = 0;
00212     while (arr[hash] != NULL && arr[hash]->key != key) {
00213         hash = _wrap(hash + (++i * i + i) / 2, capacity);
00214     }
00215
00216     arr[hash] = temp;
00217 }
00218
00219 template <typename K, typename V> long long *HashMap<K, V>::_getPows() {
00220     long long mod = 10e9 + 7;
00221     long long pow = 1;
00222     long long *pows = new long long[25];
00223     for (int i = 0; i < 25; i++) {
00224         pows[i] = pow;
00225         pow = (pow * 5503) % mod;
00226     }
00227     return pows;
00228 }
00229
00230 template <typename K, typename V>
00231 int HashMap<K, V>::_wrap(long long key, int size) {
00232     key %= size;
00233     if (key < 0)
00234         key += size;
00235     return key;
00236 }
00237
00238 template <typename K, typename V>
00239 std::string HashMap<K, V>::_toStringGeneric(const K &val) {
00240     std::ostringstream oss;
00241     oss << val;
00242     return oss.str();
00243 }
00244
00245 template <typename K, typename V> int HashMap<K, V>::_hashCode(K key) {
00246     long long mod = 10e9 + 9;
00247     long long hash = 0;
00248     long long pow = 1;
00249     int i = 0;
00250     for (char c : _toStringGeneric(key)) {
00251         hash = (hash + c * pow) % mod;
00252         i++;
00253         pow = pows[i];
00254     }
00255     return _wrap(hash, capacity);
00256 }
00257
00258 template <typename K, typename V> double HashMap<K, V>::_getLoadFactor() {
00259     return (double)size / capacity;
00260 }
00261
00262 template <typename K, typename V> void HashMap<K, V>::insert(K key, V value) {
00263     if (_getLoadFactor() > 0.6) {
00264         _grow();
00265     }
00266     _insert(key, value, table);
00267     size++;
00268 }
00269
00270 template <typename K, typename V> bool HashMap<K, V>::containsKey(K key) {
00271
00272     int hash = _hashCode(key);
00273
00274     int i = 0;
00275     while (table[hash] != NULL) {
00276         if (table[hash]->key == key)
00277             return true;
00278         hash = _wrap(hash + (++i * i + i) / 2, capacity);
00279     }

```

```

00280
00281     return false;
00282 }
00283
00284 template <typename K, typename V>
00285 std::optional<V> HashMap<K, V>::remove(K key) {
00286     int hash = _hashCode(key);
00287
00288     int i = 0;
00289     while (table[hash] != NULL) {
00290         if (table[hash]->key == key) {
00291             HashNode<K, V> *temp = table[hash];
00292             table[hash] = nullptr;
00293             size--;
00294             return temp->value;
00295         }
00296         hash = _wrap(hash + (++i * i + i) / 2, capacity);
00297     }
00298     return std::nullopt;
00299 }
00300
00301 }
00302
00303 template <typename K, typename V> std::optional<V> HashMap<K, V>::get(K key) {
00304     int hashIndex = _hashCode(key);
00305
00306     int i = 0;
00307     while (table[hashIndex] != NULL) {
00308         if (table[hashIndex]->key == key)
00309             return table[hashIndex]->value;
00310         hashIndex = _wrap(hashIndex + (++i * i + i) / 2, capacity);
00311     }
00312     return std::nullopt;
00313 }
00314
00315 template <typename K, typename V> int HashMap<K, V>::getSize() { return size; }
00316
00317 template <typename K, typename V> bool HashMap<K, V>::isEmpty() {
00318     return size == 0;
00319 }
00320
00321 template <typename K, typename V> HashNode<K, V> **HashMap<K, V>::toList() {
00322     HashNode<K, V> **list = new HashNode<K, V> *[size];
00323     int index = 0;
00324     for (int i = 0; i < capacity; i++) {
00325         if (table[i] != NULL) {
00326             list[index++] = table[i];
00327         }
00328     }
00329     return list;
00330 }
00331
00332 template <typename K, typename V> void HashMap<K, V>::display() {
00333     for (int i = 0; i < capacity; i++) {
00334         if (table[i] != NULL)
00335             std::cout << "key = " << table[i]->key << " value = " << table[i]->value
00336                 << std::endl;
00337     }
00338 }
00339
00340 template <typename K, typename V> void HashMap<K, V>::clear() {
00341     for (int i = 0; i < capacity; i++) {
00342         if (table[i] != NULL) {
00343             delete table[i];
00344             table[i] = NULL;
00345         }
00346     }
00347     size = 0;
00348 }

```

4.5. InvertedIndex.cpp

```

00001 #include "InvertedIndex.hpp"
00002
00003 InvertedIndex::InvertedIndex(int initialCapacity) : map_(initialCapacity) {}
00004
00005 InvertedIndex::~InvertedIndex() { clear(); }
00006
00007 void InvertedIndex::add(const std::string &key, int rowId) {
00008     if (map_.containsKey(key)) {
00009         std::vector<int> existing = map_.get(key).value();
00010         existing.push_back(rowId);
00011         map_.insert(key, existing);

```

```

00012     } else {
00013         std::vector<int> vec;
00014         vec.push_back(rowId);
00015         map_.insert(key, vec);
00016     }
00017 }
00018
00019 std::vector<int> InvertedIndex::get(const std::string &key) {
00020     if (map_.containsKey(key)) {
00021         return map_.get(key).value();
00022     } else {
00023         return {};
00024     }
00025 }
00026
00027 void InvertedIndex::remove(const std::string &key, int rowId) {
00028     if (!map_.containsKey(key))
00029         return;
00030
00031     std::vector<int> existing = map_.get(key).value();
00032
00033     std::vector<int> temp;
00034     for (int x : existing) {
00035         if (x != rowId) {
00036             temp.push_back(x);
00037         }
00038     }
00039     existing = temp;
00040
00041     if (existing.empty()) {
00042         map_.remove(key);
00043     } else {
00044         map_.insert(key, existing);
00045     }
00046 }
00047
00048 void InvertedIndex::clear() { map_.clear(); }

```

4.6. Referencia del archivo src/structures/InvertedIndex.hpp

Implementación de un índice invertido utilizando un mapa hash.

```

#include "HashMap.hpp"
#include <string>
#include <vector>

```

Clases

- class `InvertedIndex`

Índice invertido basado en [HashMap](#) para asociar claves con listas de identificadores.

4.6.1. Descripción detallada

Implementación de un índice invertido utilizando un mapa hash.

Esta clase permite asociar claves (palabras, términos, etc.) con listas de identificadores de filas, facilitando búsquedas eficientes de ocurrencias de términos en colecciones de datos.

4.7. InvertedIndex.hpp

[Ir a la documentación de este archivo.](#)

```

00001
00009
00010 #include "HashMap.hpp"
00011 #include <string>
00012 #include <vector>
00013
00022 class InvertedIndex {
00023 public:
00029     InvertedIndex(int initialCapacity = 101);
00030
00034     ~InvertedIndex();
00035
00041     void add(const std::string &key, int rowId);
00042
00049     std::vector<int> get(const std::string &key);
00050
00057     void remove(const std::string &key, int rowId);
00058
00062     void clear();
00063
00064 private:
00065     HashMap<std::string, std::vector<int>>
00066         map_;
00068 };

```

4.8. Referencia del archivo src/system/IndexEntry.hpp

Definición de la estructura [IndexEntry](#) para índices de tablas.

```

#include <string>
#include <variant>

```

Clases

- struct [IndexEntry](#)
Entrada de índice para estructuras de índice (B+ o invertido).

typedefs

- using **Cell** = std::variant<int, double, std::string>
Representa el valor de una celda, que puede ser int, double o string.

4.8.1. Descripción detallada

Definición de la estructura [IndexEntry](#) para índices de tablas.

Contiene la clave (valor de celda) y el identificador de fila correspondiente, junto con operadores de comparación.

4.9. IndexEntry.hpp

[Ir a la documentación de este archivo.](#)

```
00001
00008
00009 #pragma once
00010 #include <string>
00011 #include <variant>
00012
00013 using Cell = std::variant<int, double, std::string>;
00014
00022 struct IndexEntry {
00023     Cell key;
00024     int rowId;
00025
00030     bool operator<(const IndexEntry &other) const;
00035     bool operator>(const IndexEntry &other) const;
00040     bool operator<=(const IndexEntry &other) const;
00045     bool operator>=(const IndexEntry &other) const;
00050     bool operator==(const IndexEntry &other) const;
00051 };
```

4.10. Referencia del archivo src/system/Table.cpp

Implementación de la clase [Table](#).

```
#include "Table.hpp"
#include "IndexEntry.hpp"
#include <climits>
#include <iostream>
#include <sstream>
#include <stdexcept>
```

4.10.1. Descripción detallada

Implementación de la clase [Table](#).

La clase [Table](#) representa una tabla de base de datos simple. Permite definir un esquema, insertar filas, crear índices numéricos y de texto, y realizar búsquedas indexadas.

4.11. Table.cpp

[Ir a la documentación de este archivo.](#)

```
00001 #include "Table.hpp"
00002 #include "IndexEntry.hpp"
00003 #include <climits>
00004 #include <iostream>
00005 #include <sstream>
00006 #include <stdexcept>
00007
00015
00016 Table::Table(const std::vector<std::pair<std::string, DataType> &cols,
00017             int hashCapacity)
00018     : nameToIndex_(hashCapacity), bptIndices_(hashCapacity),
00019       invIndices_(hashCapacity) {
00020
00021     for (int i = 0; i < cols.size(); ++i) {
00022         const auto &[colName, colType] = cols[i];
00023         if (colName.empty()) {
00024             throw std::invalid_argument("Column name cannot be empty");
00025         }
00026     }
00027 }
```

```

00026
00027     if (nameToIndex_.containsKey(colName)) {
00028         throw std::invalid_argument("Duplicate column name: " + colName);
00029     }
00030     schema_.push_back({colName, colType});
00031     nameToIndex_.insert(colName, i);
00032 }
00033 }
00034
00035 Table::~Table() {
00036
00037     for (int i = 0; i < bptIndices_.getSize(); ++i) {
00038     }
00039
00040     for (int i = 0; i < schema_.size(); ++i) {
00041         const auto &colName = schema_[i].name;
00042         if (bptIndices_.containsKey(colName)) {
00043             BPlusTree<IndexEntry> *treePtr = bptIndices_.get(colName).value();
00044             delete treePtr;
00045         }
00046         if (invIndices_.containsKey(colName)) {
00047             InvertedIndex *idxPtr = invIndices_.get(colName).value();
00048             delete idxPtr;
00049         }
00050     }
00051 }
00052
00053 bool Table::cellMatchesType(const Cell &cell, DataType dt) {
00054     switch (dt) {
00055     case DataType::INT:
00056         return std::holds_alternative<int>(cell);
00057     case DataType::DOUBLE:
00058         return std::holds_alternative<double>(cell);
00059     case DataType::STRING:
00060         return std::holds_alternative<std::string>(cell);
00061     }
00062     return false;
00063 }
00064
00065 std::string Table::dataTypeToString(DataType dt) {
00066     switch (dt) {
00067     case DataType::INT:
00068         return "INT";
00069     case DataType::DOUBLE:
00070         return "DOUBLE";
00071     case DataType::STRING:
00072         return "STRING";
00073     }
00074     return "UNKNOWN";
00075 }
00076
00077 int Table::columnIndex(const std::string &colName) {
00078     if (!nameToIndex_.containsKey(colName)) {
00079         throw std::invalid_argument("Unknown column '" + colName + "'");
00080     }
00081     return nameToIndex_.get(colName).value();
00082 }
00083
00084 void Table::printSchema() const {
00085     std::cout << "Schema:\n";
00086     for (const auto &col : schema_) {
00087         std::cout << " " << col.name << " : " << dataTypeToString(col.type)
00088             << "\n";
00089     }
00090 }
00091
00092 void Table::printAllRows() const {
00093     for (const auto &col : schema_) {
00094         std::cout << col.name << "\t";
00095     }
00096     std::cout << "\n-----\n";
00097
00098     for (int r = 0; r < data_.size(); ++r) {
00099         const auto &row = data_[r];
00100         for (int c = 0; c < row.size(); ++c) {
00101             std::visit([&auto &&val] { std::cout << val << "\t"; }, row[c]);
00102         }
00103         std::cout << "\n";
00104     }
00105 }
00106
00107 Cell Table::getCell(int rowIndex, const std::string &colName) {
00108     int cid = columnIndex(colName);
00109     if (rowIndex >= data_.size()) {
00110         throw std::out_of_range("Row index out of range");
00111     }
00112 }

```

```

00116     return data_[rowIndex][cidx];
00117 }
00118
00119 void Table::insertRow(const std::vector<Cell> &row) {
00120     if (row.size() != schema_.size()) {
00121         throw std::invalid_argument("Row size does not match schema size");
00122     }
00123
00124     for (int i = 0; i < row.size(); ++i) {
00125         if (!cellMatchesType(row[i], schema_[i].type)) {
00126             std::ostringstream oss;
00127             oss << "Type mismatch for column '" << schema_[i].name << "' at index "
00128                 << i;
00129             throw std::invalid_argument(oss.str());
00130         }
00131     }
00132
00133     int newRowId = data_.size();
00134     data_.push_back(row);
00135
00136     for (int i = 0; i < schema_.size(); ++i) {
00137         const std::string &colName = schema_[i].name;
00138         if (bptIndices_.containsKey(colName)) {
00139             BPlusTree<IndexEntry> *treePtr = bptIndices_.get(colName).value();
00140             IndexEntry ie(row[i], newRowId);
00141             treePtr->insert(ie);
00142         }
00143     }
00144
00145     for (int i = 0; i < schema_.size(); ++i) {
00146         if (schema_[i].type == DataType::STRING) {
00147             const std::string &colName = schema_[i].name;
00148             if (invIndices_.containsKey(colName)) {
00149                 InvertedIndex *invPtr = invIndices_.get(colName).value();
00150                 const std::string &keyStr = std::get<std::string>(row[i]);
00151                 invPtr->add(keyStr, newRowId);
00152             }
00153         }
00154     }
00155 }
00156
00157 void Table::createBPlusTreeIndex(const std::string &colName, int degree) {
00158     int cidx = columnIndex(colName);
00159     DataType dt = schema_[cidx].type;
00160
00161     BPlusTree<IndexEntry> *treePtr = new BPlusTree<IndexEntry>(degree);
00162
00163     for (int rid = 0; rid < data_.size(); ++rid) {
00164         const Cell &cell = data_[rid][cidx];
00165         IndexEntry ie(cell, rid);
00166         treePtr->insert(ie);
00167     }
00168
00169     if (bptIndices_.containsKey(colName)) {
00170         delete bptIndices_.get(colName).value();
00171     }
00172     bptIndices_.insert(colName, treePtr);
00173 }
00174
00175 void Table::createInvertedIndex(const std::string &colName) {
00176     int cidx = columnIndex(colName);
00177     if (schema_[cidx].type != DataType::STRING) {
00178         throw std::invalid_argument("Inverted index only valid on STRING columns");
00179     }
00180
00181     InvertedIndex *invPtr = new InvertedIndex(/*initialCapacity=*/101);
00182
00183     for (int rid = 0; rid < data_.size(); ++rid) {
00184         const std::string &val = std::get<std::string>(data_[rid][cidx]);
00185         invPtr->add(val, rid);
00186     }
00187
00188     if (invIndices_.containsKey(colName)) {
00189         delete invIndices_.get(colName).value();
00190     }
00191     invIndices_.insert(colName, invPtr);
00192 }
00193
00194 std::vector<int> Table::searchByIndex(const std::string &colName,
00195                                     const Cell &key) {
00196     int cidx = columnIndex(colName);
00197     DataType dt = schema_[cidx].type;
00198
00199     if (bptIndices_.containsKey(colName)) {
00200         BPlusTree<IndexEntry> *treePtr = bptIndices_.get(colName).value();
00201
00202         int maxPossible = data_.size();

```

```

00203     IndexEntry *buffer = new IndexEntry[maxPossible];
00204
00205     IndexEntry start{key, 0};
00206     IndexEntry end{key, INT_MAX};
00207
00208     int foundCount = treePtr->searchRange(start, end, buffer, (int)maxPossible);
00209
00210     std::vector<int> result;
00211     result.reserve(foundCount);
00212     for (int i = 0; i < foundCount; ++i) {
00213         result.push_back(buffer[i].rowId);
00214     }
00215     delete[] buffer;
00216     return result;
00217 }
00218
00219 if (schema_[cidx].type == DataType::STRING &&
00220     invIndices_.containsKey(colName)) {
00221     InvertedIndex *invPtr = invIndices_.get(colName).value();
00222     const std::string &keyStr = std::get<std::string>(key);
00223     return invPtr->get(keyStr);
00224 }
00225
00226 return {};
00227 }

```

4.12. Referencia del archivo src/system/Table.hpp

Definición de la clase [Table](#) y estructuras auxiliares para la gestión de tablas de datos.

```

#include "IndexEntry.hpp"
#include "../structures/BPlusTree.hpp"
#include "../structures/InvertedIndex.cpp"
#include <string>
#include <variant>
#include <vector>

```

Clases

- struct [Column](#)
Representa una columna de la tabla, con nombre y tipo de dato.
- class [Table](#)
Representa una tabla de datos con soporte para índices y operaciones básicas.

typedefs

- using [Cell](#) = std::variant<int, double, std::string>

Enumeraciones

- enum class [DataType](#) { [INT](#) , [DOUBLE](#) , [STRING](#) }
Tipos de datos soportados por las columnas de la tabla.

4.12.1. Descripción detallada

Definición de la clase [Table](#) y estructuras auxiliares para la gestión de tablas de datos.

Proporciona una estructura de tabla relacional con soporte para índices B+ y de índice invertido, permitiendo operaciones de inserción, búsqueda e impresión de datos.

4.13. Table.hpp

[Ir a la documentación de este archivo.](#)

```

00001
00008
00009 #include "IndexEntry.hpp"
00010 #include "../structures/BPlusTree.hpp"
00011 #include "../structures/InvertedIndex.cpp"
00012 #include <string>
00013 #include <variant>
00014 #include <vector>
00015
00020 enum class DataType { INT, DOUBLE, STRING };
00021
00026 using Cell = std::variant<int, double, std::string>;
00027
00032 struct Column {
00033     std::string name;
00034     DataType type;
00035 };
00036
00044 class Table {
00045 public:
00051     Table(const std::vector<std::pair<std::string, DataType>> &cols,
00052           int hashCapacity = 101);
00053
00057     ~Table();
00058
00063     void insertRow(const std::vector<Cell> &row);
00064
00068     void printSchema() const;
00069
00073     void printAllRows() const;
00074
00081     Cell getCell(int rowIndex, const std::string &colName);
00082
00088     void createBPlusTreeIndex(const std::string &colName, int degree);
00089
00094     void createInvertedIndex(const std::string &colName);
00095
00102     std::vector<int> searchByIndex(const std::string &colName, const Cell &key);
00103
00104 private:
00105     std::vector<Column> schema_;
00106     HashMap<std::string, int> nameToIndex_;
00107     std::vector<std::vector<Cell>> data_;
00108
00109     HashMap<std::string, BPlusTree<IndexEntry> *> bptIndices_;
00110     HashMap<std::string, InvertedIndex *> invIndices_;
00111
00118     static bool cellMatchesType(const Cell &cell, DataType dt);
00119
00125     static std::string dataTypeToString(DataType dt);
00126
00132     int columnIndex(const std::string &colName);
00133 };

```

Índice alfabético

- [_findIndex](#)
BPlusTree< T >, [7](#)
 - [_findKey](#)
BPlusTree< T >, [7](#)
 - [_findRange](#)
BPlusTree< T >, [7](#)
 - [_getLoadFactor](#)
HashMap< K, V >, [15](#)
 - [_getPows](#)
HashMap< K, V >, [15](#)
 - [_hashCode](#)
HashMap< K, V >, [15](#)
 - [_innerInsert](#)
BPlusTree< T >, [8](#)
 - [_insert](#)
HashMap< K, V >, [16](#)
 - [_insertInChild](#)
BPlusTree< T >, [8](#)
 - [_insertInParent](#)
BPlusTree< T >, [8](#)
 - [_insertItem](#)
BPlusTree< T >, [9](#)
 - [_print](#)
BPlusTree< T >, [9](#)
 - [_removeFromParent](#)
BPlusTree< T >, [9](#)
 - [_toStringGeneric](#)
HashMap< K, V >, [16](#)
 - [_wrap](#)
HashMap< K, V >, [16](#)
- [~BPlusTree](#)
BPlusTree< T >, [6](#)
- [~HashMap](#)
HashMap< K, V >, [15](#)
- [~InvertedIndex](#)
InvertedIndex, [22](#)
- [~Table](#)
Table, [26](#)
- [add](#)
InvertedIndex, [22](#)
- [BPlusTree](#)
BPlusTree< T >, [6](#)
- [BPlusTree< T >](#), [5](#)
 - [_findIndex](#), [7](#)
 - [_findKey](#), [7](#)
 - [_findRange](#), [7](#)
 - [_innerInsert](#), [8](#)
 - [_insertInChild](#), [8](#)
 - [_insertInParent](#), [8](#)
 - [_insertItem](#), [9](#)
 - [_print](#), [9](#)
 - [_removeFromParent](#), [9](#)
 - [_toStringGeneric](#), [16](#)
 - [_wrap](#), [16](#)
- [cellMatchesType](#)
Table, [26](#)
- [clear](#)
BPlusTree< T >, [11](#)
- [Column](#), [13](#)
- [columnIndex](#)
Table, [26](#)
- [containsKey](#)
HashMap< K, V >, [17](#)
- [createBPlusTreeIndex](#)
Table, [27](#)
- [createInvertedIndex](#)
Table, [27](#)
- [dataTypeToString](#)
Table, [27](#)
- [get](#)
HashMap< K, V >, [17](#)
InvertedIndex, [22](#)
- [getCell](#)
Table, [27](#)
- [getRoot](#)
BPlusTree< T >, [11](#)
- [getSize](#)
HashMap< K, V >, [17](#)
- [HashMap](#)
HashMap< K, V >, [14](#)
- [HashMap< K, V >](#), [13](#)
 - [_getLoadFactor](#), [15](#)
 - [_getPows](#), [15](#)
 - [_hashCode](#), [15](#)
 - [_insert](#), [16](#)
 - [_toStringGeneric](#), [16](#)
 - [_wrap](#), [16](#)
- [~HashMap](#), [15](#)

- containsKey, [17](#)
- get, [17](#)
- getSize, [17](#)
- HashMap, [14](#)
- insert, [17](#)
- isEmpty, [18](#)
- remove, [18](#)
- toList, [18](#)
- HashNode
 - HashNode< K, V >, [19](#)
- HashNode< K, V >, [19](#)
 - HashNode, [19](#)
- IndexEntry, [20](#)
 - operator<, [20](#)
 - operator<=, [20](#)
 - operator>, [21](#)
 - operator>=, [21](#)
 - operator==, [20](#)
- insert
 - BPlusTree< T >, [11](#)
 - HashMap< K, V >, [17](#)
- insertRow
 - Table, [28](#)
- InvertedIndex, [21](#)
 - ~InvertedIndex, [22](#)
 - add, [22](#)
 - get, [22](#)
 - InvertedIndex, [22](#)
 - remove, [23](#)
- isEmpty
 - HashMap< K, V >, [18](#)
- Node
 - Node< T >, [24](#)
- Node< T >, [23](#)
 - Node, [24](#)
- operator<
 - IndexEntry, [20](#)
- operator<=
 - IndexEntry, [20](#)
- operator>
 - IndexEntry, [21](#)
- operator>=
 - IndexEntry, [21](#)
- operator==
 - IndexEntry, [20](#)
- remove
 - BPlusTree< T >, [11](#)
 - HashMap< K, V >, [18](#)
 - InvertedIndex, [23](#)
- search
 - BPlusTree< T >, [12](#)
- searchByIndex
 - Table, [28](#)
- searchRange
 - BPlusTree< T >, [12](#)
 - src/structures/BPlusTree.hpp, [29](#)
 - src/structures/HashMap.hpp, [38](#), [39](#)
 - src/structures/InvertedIndex.cpp, [41](#)
 - src/structures/InvertedIndex.hpp, [42](#), [43](#)
 - src/system/IndexEntry.hpp, [43](#), [44](#)
 - src/system/Table.cpp, [44](#)
 - src/system/Table.hpp, [47](#), [48](#)
- Table, [24](#)
 - ~Table, [26](#)
 - cellMatchesType, [26](#)
 - columnIndex, [26](#)
 - createBPlusTreeIndex, [27](#)
 - createInvertedIndex, [27](#)
 - dataTypeToString, [27](#)
 - getCell, [27](#)
 - insertRow, [28](#)
 - searchByIndex, [28](#)
 - Table, [26](#)
- toList
 - HashMap< K, V >, [18](#)